



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

MULTITHREADING IN GUI

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

PARAMVIR SINGH



AGENDA

- *Code Demo (last lecture)*
- Thread synchronization
- Multi-threaded Singleton
- Multithreading in GUIs
- *Code resources (Canvas)*
 - *PrimeFactorisation App*
 - *ThumbnailGeneratorApp*

THREAD SYNCHRONIZATION

THE *SYNCHRONIZED* METHOD

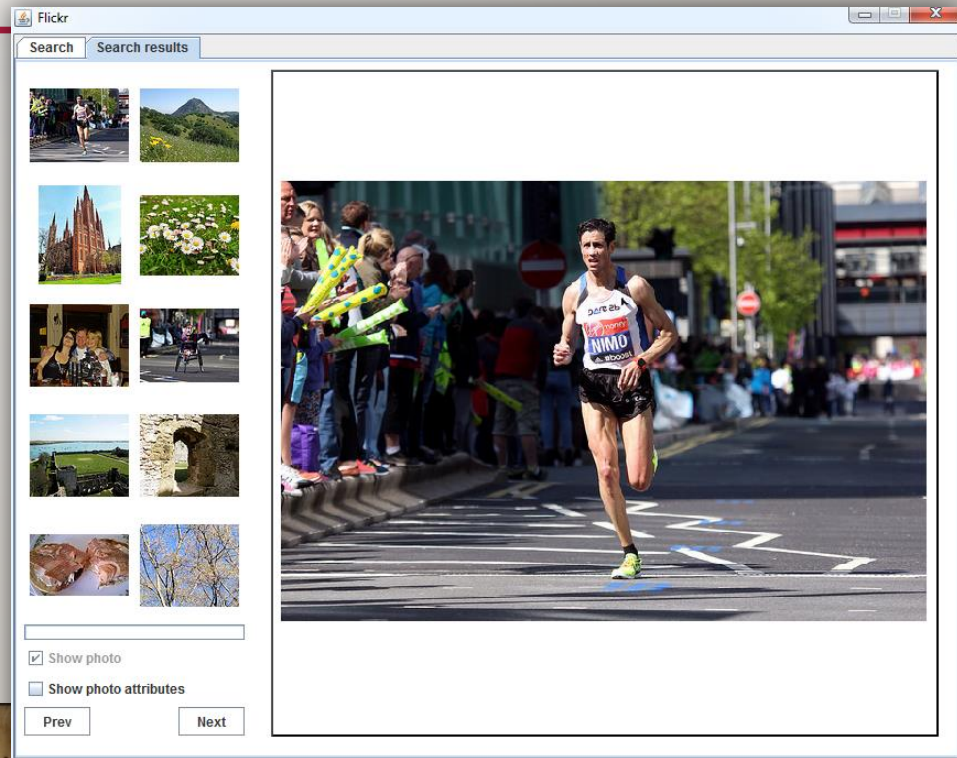
- Synchronizing the execution of two or more threads could **solve the problem of non-deterministic outputs** of multithreaded programs.
- The **synchronized** keyword can be declared in different ways. It can be used in a method name.
- Using *synchronized* in this context allows only **one thread at a time** to execute this method.
- Any other threads attempting to access this method will be placed into the **blocked** state. Note the difference between **blocked** and **waiting** is evident in this case.
- Two different examples >>

```
public class SynchronizedCounter {  
  
    int value = 0;  
  
    public synchronized void increment(String incrementor) {  
        this.value += 1;  
    }  
}
```

```
public synchronized void send(String msg)  
{  
    System.out.println("Fast \t" + msg );  
    try  
    {  
        Thread.sleep(1000);  
    }  
    catch (Exception e)  
    {  
        System.out.println("Thread was interrupted.");  
    }  
    System.out.println("\n" + msg + "Sent");  
}
```

MULTITHREADING IN GUIs

- Multithreading is necessary in any non-trivial GUI application
 - If a single thread is used to handle GUI events and to execute time-consuming tasks, the GUI will appear to users to freeze and be unresponsive
- Introducing multiple threads brings complexity and the need to protect objects from concurrent access
 - GUI components are typically **not thread-safe** – and care must be taken when accessing them

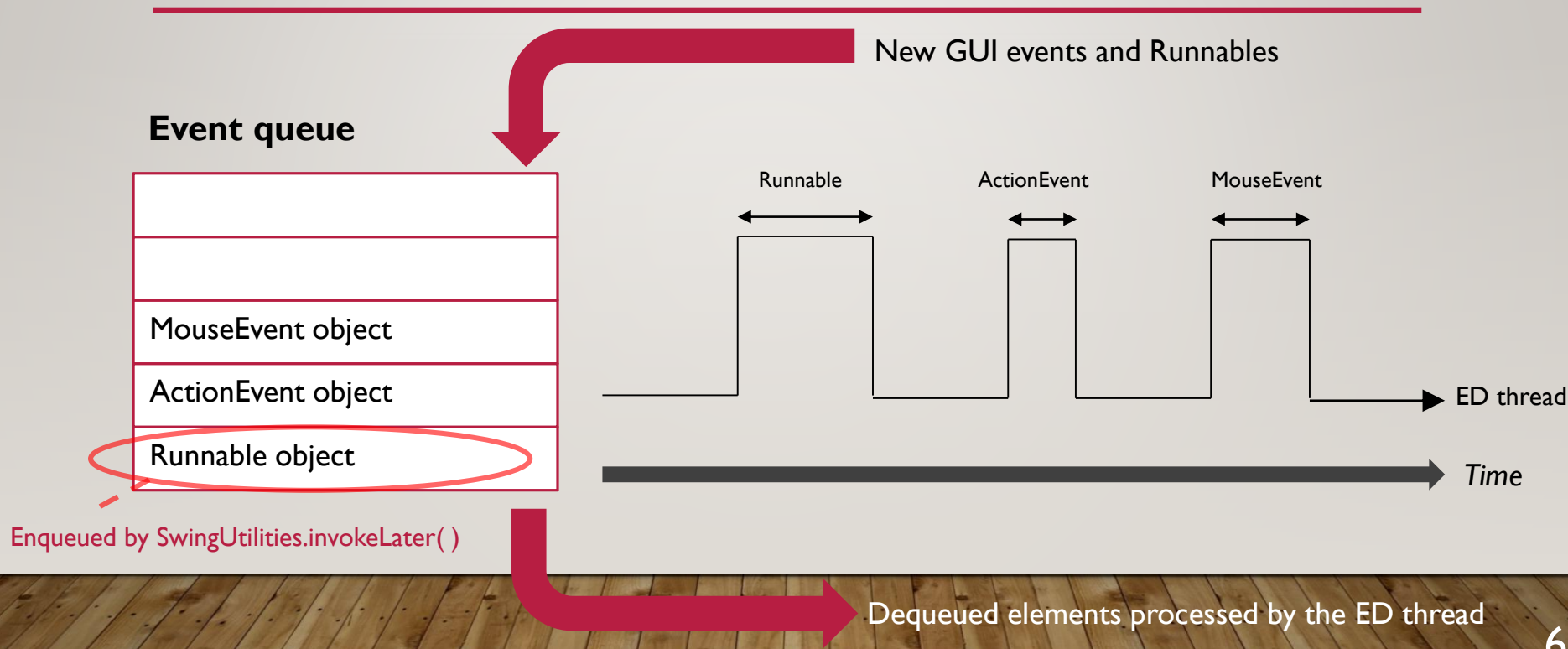


MULTITHREADING IN SWING

- There are **three types of threads** in a Swing app
 - **Initial thread**
 - Executes initial application code, e.g. main()
 - **Event Dispatch (ED) thread**
 - Responsible for all GUI event handling
 - All code that interacts with the Swing framework should execute on this thread
 - **Worker thread(s)**
 - AKA background threads, they execute time-consuming tasks
- These threads are created and managed by the Java run-time or Swing framework

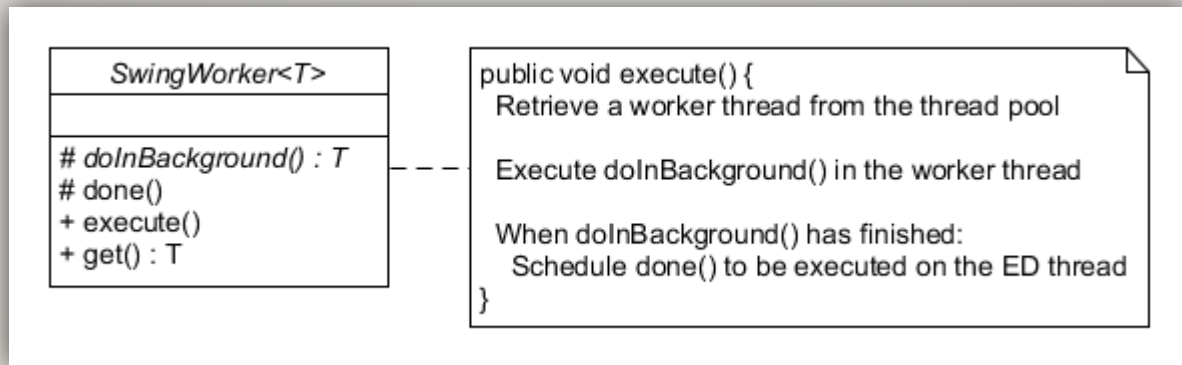
```
public class MySwingApp extends JPanel {  
    private static void createAndShowGUI() {  
        JFrame frame = new JFrame( );  
  
        JComponent newContentPane = new MySwingApp( );  
        frame.add(newContentPane);  
  
        frame.pack();  
        frame.setVisible(true);  
    }  
  
    public static void main(String[ ] args) {  
        // Schedule a job for the event-dispatching thread:  
        javax.swing.SwingUtilities.invokeLater(new Runnable( ) {  
            public void run() {  
                createAndShowGUI( );  
            }  
        });  
    }  
}
```


THE *ED* THREAD



SWINGWORKER

- **SwingWorker** simplifies the development of **Swing** applications that need to perform background processing



- The **Swing** framework handles coordination between EDT and worker threads

EXAMPLE: PRIME FACTORS

```

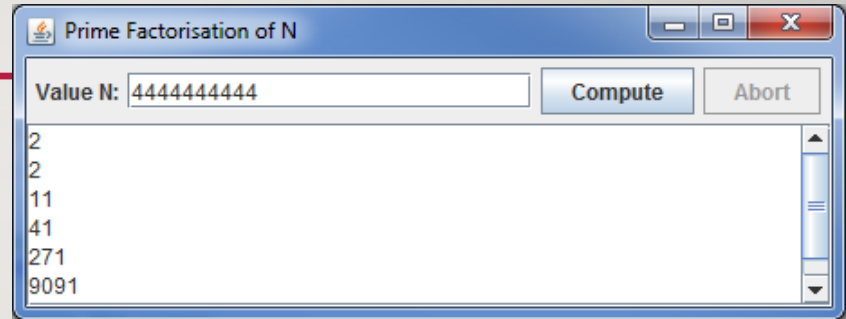
public class PrimeFactorsApp extends JPanel {
    private static void createAndShowGUI() {
        JFrame frame = new JFrame( );

        JComponent newContentPane = new PrimeFactorsApp( );
        frame.add(newContentPane);

        frame.pack( );
        frame.setVisible(true);
    }

    public static void main(String[ ] args) {
        // Schedule a job for the event-dispatching thread:
        javax.swing.SwingUtilities.invokeLater(new Runnable( ) {
            public void run() {
                createAndShowGUI();
            }
        });
    }
}

```



```

public PrimeFactorsApp( ) {
    Create GUI components:
        JButton to start the computation
        JButton to abort the computation
        JTextField to enter the value of N
        JTextArea to display results
    Layout components

    Add an ActionListener to the start button:
        Retrieve value from textfield
        Execute the computation algorithm in the ED thread
        Display results
}

```


EXAMPLE: PRIME FACTORS

```

public class PrimeFactorsApp extends JPanel {
    private static void createAndShowGUI() {
        JFrame frame = new JFrame( );

        JComponent newContentPane = new PrimeFactorsApp( );
        frame.add(newContentPane);

        frame.pack( );
        frame.setVisible(true);
    }

    public static void main(String[ ] args) {
        // Schedule a job for the event-dispatching thread:
        javax.swing.SwingUtilities.invokeLater(new Runnable( ) {
            public void run() {
                createAndShowGUI();
            }
        });
    }
}

```

```

public class PrimeFactorisationWorker extends
SwingWorker<List<Long>> {
    protected List<Long> doInBackground() {
        // Execute algorithm, build and return list of factors.
    }

    protected void done() {
        // Call get() to get the list of factors.
        // For each factor, add it to the JTextArea.
    }
}

```

```

public PrimeFactorsApp( ) {
    Create GUI components:
        JButton to start the computation
        JButton to abort the computation
        JText-field to enter the value of N
        JText-area to display results
    Layout components

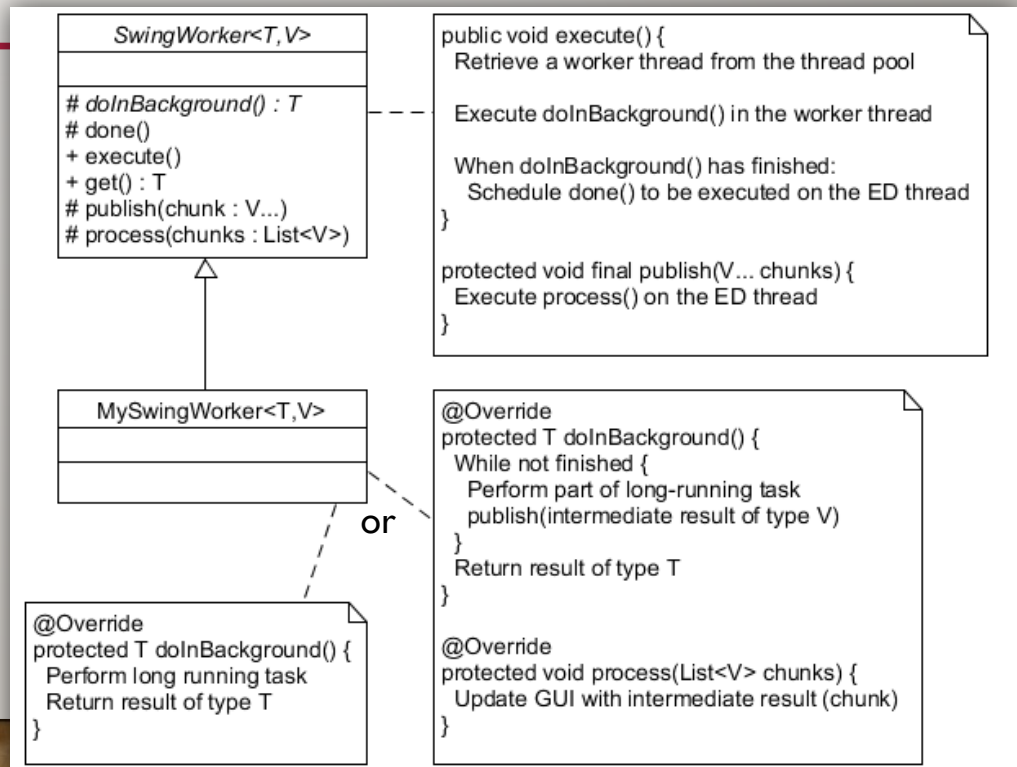
    Add an ActionListener to the start button:
        Retrieve value from textfield
        Execute the computation algorithm in the ED thread
        Display results
}

```

SWINGWORKER

PUBLISH() AND PROCESS()

- This is what SwingWorker really looks like (the previous slides were a slight simplification).
- This diagrams shows that:
 - SwingWorkers can return intermediate (partial) results during a long running computation (generic type argument V).
 - Intermediate results:
 - publish()** is a template method that is intended to be called from within **doInBackground()** (running in the worker thread).
 - publish()** arranges for **process()** to be executed by the ED thread – so that it can safely update the UI.

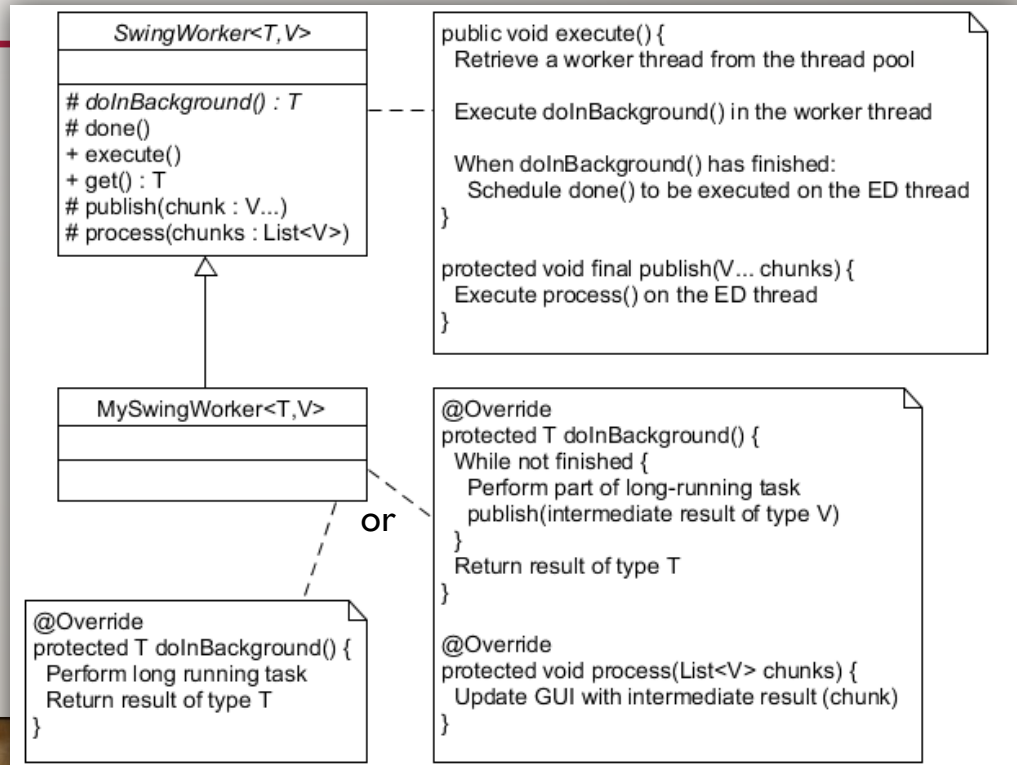


SWINGWORKER

TEMPLATE METHOD PATTERN

Template Method pattern

- The methods **execute()**, **publish()**, and **get()** are template methods, holding the "common" code.
- The overridable methods **doInBackground()**, **process()**, and **done()** serve as hook methods, in which client code provides the (variable) details for the various "steps".



EXAMPLE: PRIMEFACTORISATIONWORKER CLASS

```
public class PrimeFactorisationWorker extends
    SwingWorker<List<Long>, Long> {
```

@Override

```
protected List<Long> doInBackground() {
```

```
...
```

```
for( long i = 2; ... )
```

```
while( ... ) {
```

```
...
```

```
publish( i );
```

```
}
```

```
}
```

```
}
```

@Override

```
protected void process(List<Long> factors) {
```

```
for(Long factor : factors) {
```

```
    Append factor to JTextArea
```

```
}
```

```
...
```

```
}
```

```
if (isCancelled( )) {
    return null;
}
```

```
public class PrimeFactorsApp extends JPanel {
    private JButton _abortButton, _startButton;
    private SwingWorker<List<Long>, Long> _worker;
    ...
```

```
public PrimeFactorsApp( ) {
```

```
...
```

```
_startButton.addActionListener( new ActionListener( ) {
```

@Override

```
public void actionPerformed(ActionEvent event) {
```

```
...
```

```
_worker = new PrimeFactorisationWorker(...);
```

```
_worker.execute( );
```

```
}
```

```
});
```

```
_abortButton.addActionListener( new ActionListener( ) {
```

@Override

```
public void actionPerformed(ActionEvent event) {
```

```
_worker.cancel( );
```

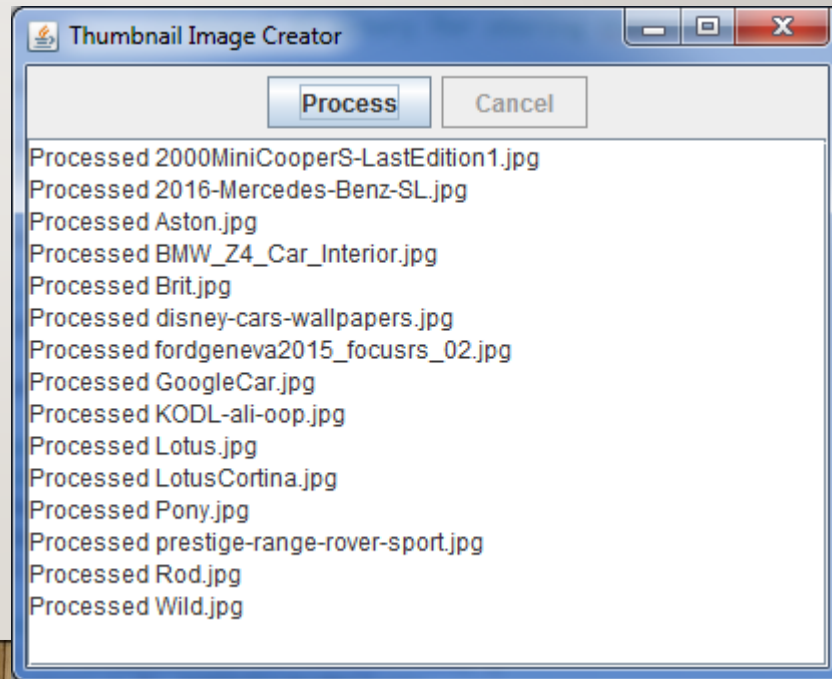
```
}
```

```
});
```

```
}
```

SUGGESTED TASK

- **ThumbnailGeneratorApp** is a single-threaded Swing app that converts images in a selected directory to thumbnails
- Create a new version of the app to more appropriately use a **SwingWorker** to load and generate the thumbnails
 - Enhancements include: cancellable processing, displaying intermediate results, and showing a progress bar



REVIEW

- **Learnt something about:**
 - **Threading in GUIs**
 - Expensive tasks must not be processed on the event-dispatch (ED) thread because the ED thread would be unavailable to process GUI events
 - **Swing's solution for GUI multithreading**
 - Using `SwingWorker`, the Swing framework manages background processing using worker threads and coordination of workers with the ED thread
 - Notable properties of `SwingWorker` include:
 - Provision for safe consumption of background processing results on the ED thread
 - Use of generic type arguments to provide for type-safety of results
 - Support for both intermediate and final results
 - Bound properties whose state changes are processed on the ED thread
 - Application of the Template Method and Listener patterns

REFERENCES

- For more information, see **Oracle's SwingWorker tutorial**:
<http://docs.oracle.com/javase/tutorial/uiswing/concurrency>